## Introduction:

In Malaysia, businesses employing foreign workers face severe fines of up to RM50,000 per worker or operational shutdowns for simple paperwork errors. Currently, most companies manage complex permit renewals and medical checks using error-prone Excel spreadsheets. *PermitIQ* solves this problem as an AI-powered software system that automatically extracts data from messy worker documents via multi-modal vision, tracks critical government deadlines, and utilizes a multi-agent orchestration engine to tell HR managers exactly what compliance obligations they need to fulfill.

## Purpose:

The system analysis document highlights the technical scope, system boundaries, and design-related decisions behind the development of PermitIQ.

The key elements of the system that have been covered are as follows:

## Architecture:

It is a cloud-native platform structured across multiple layers. The services are separated into distinct environments, including a React frontend (Vite + Tailwind), Python FastAPI microservices, a Multi-Agent Reasoning Engine built on *LangGraph* powered by *Google's Gemini* models, and a *Firebase Firestore (NoSQL)* backend for real-time state management and document storage.

## Data Flows:

How unstructured worker data (e.g., passports, medical reports, company SSM files) flows from the HR Manager's uploads into Google Cloud Storage, moves through the *Gemini Multi-modal Document Parser*, gets validated against deterministic rules (e.g., JTKSM quota, Act 446 capacity), is persisted in Firebase Firestore, and finally pushes real-time compliance timelines and actionable alerts to the user dashboard via WebSocket/polling.

## Model Process:

The model shows the end-to-end stateful workflow of the core features: document ingestion, automated VDR (Visa Dengan Rujukan) pipeline assembly, autonomous compliance mapping based on the Multi-Tier Levy Mechanism (MTLM), and dynamic exception handling (e.g., FOMEMA failures) using a *Human-in-the-Loop (HITL) interrupt system* for high-stakes decisions.

## Role of Reference:

This document serves as the primary technical reference for all developers, prompt engineers, and testers to understand the overall high-level architecture, API contracts, LangGraph state machine transitions, and database schemas of this project.

## Background:

Malaysian SMEs, particularly in macro-critical sectors like Manufacturing and Food & Beverage, rely heavily on foreign workers. However, under the strict liability doctrine of the Immigration Act 1959/63, employing undocumented or non-compliant workers results in catastrophic fines and immediate operational shutdowns. Currently, these businesses are forced to manage fragmented compliance tasks manually across multiple unlinked government portals and shifting regulations.

**Previous Version:**

The existing system for our target market consists of manual, static Excel spreadsheets and physical filing cabinets. It relies entirely on human memory and manual data entry to track overlapping deadlines for temporary work permits (PLKS), multi-tier levies, and enhanced FOMEMA medical screenings. It lacks proactive alerts, cannot dynamically adapt to edge cases like medical failures, and leaves the SME highly vulnerable to costly administrative oversights.

**Changes in Major Architectural Components:**

The shift from a manual tracking system to PermitIQ introduces several groundbreaking new capabilities:

1. Multi-Agent Orchestration (LangGraph): Moves the system from static logic to a dynamic, graph-based orchestration layer where specialized agents (Parser, Pre-Form Validator, Signature Tracker, Strategist) actively manage workflow states sequentially.

2. Multi-Modal Document Parsing: Utilizes Gemini Vision to extract critical fields (like MRZ codes and passport numbers) directly from raw images, replacing manual data entry.

3. Deterministic Tool Execution: Utilizes Python-based compliance calculators that the LLM can independently call to solve complex legal constraints and financial math (e.g., Levy calculations, age limit checks) without hallucinating.

4. Human-in-the-Loop (HITL) Fallback: Introduces a critical safety layer where the AI pipeline automatically pauses and requests human administrative review when encountering missing fields, low-confidence OCR results, or ambiguous medical results.

## Target Stakeholder

| Stakeholders | Roles | Expectations |
|---|---|---|
| SME Owner / Director | Executive oversight and financial risk management for the company. | Clear visualization of maximum statutory fine exposure based on current data, and automated risk alerts. |
| HR / Admin Manager | Manages worker onboarding, document uploads, and daily compliance workflows. | Seamless document data extraction, automated obligation generation, and HITL dashboards to quickly resolve missing data. |
| Operations Supervisor | Tracks worker compliance status on the ground, specific to their departments. | Real-time compliance status to prevent operational halts, Halal certification revocations, or municipal premise shutdowns.. |
| Development Team | Build the platform, integrate the GLM-4 API, and maintain the microservices. | Clear API usage contracts, robust prompt service modules, and a well-documented state machine architecture. |
| QA Team | Validating the system behavior, state transitions, and AI output boundaries. | Defined scope, clear multi-agent state transition boundaries, and coverage of edge cases (e.g., HITL interruptions, FOMEMA failures, expired permits). |

## System Architecture & Design

## High Level Architecture

## Overview

| Type | Details |
|---|---|
| System | Web-Based Multi-Agent Compliance System |
| Architecture | Cloud-Server with Microservices + Agentic Workflow Architecture |

## System Description

PermitIQ is primarily structured as a **client-server-based system** consisting of three types of clients:

1. **Worker Web Application**
    - Upload passport, medical reports, and supporting documents
    - Track application status and workflow progress
2. **Employer Dashboard**
    - Submit company quota and sponsorship information
    - Monitor application status and compliance evaluation results
3. **Admin HITL Console**
    - Handle AI uncertainty cases requiring human intervention
    - Approve/reject flagged applications
    - Review audit logs and compliance decisions

All clients communicate through a **Backend API Gateway layer**. The backend is composed of multiple independent microservices:

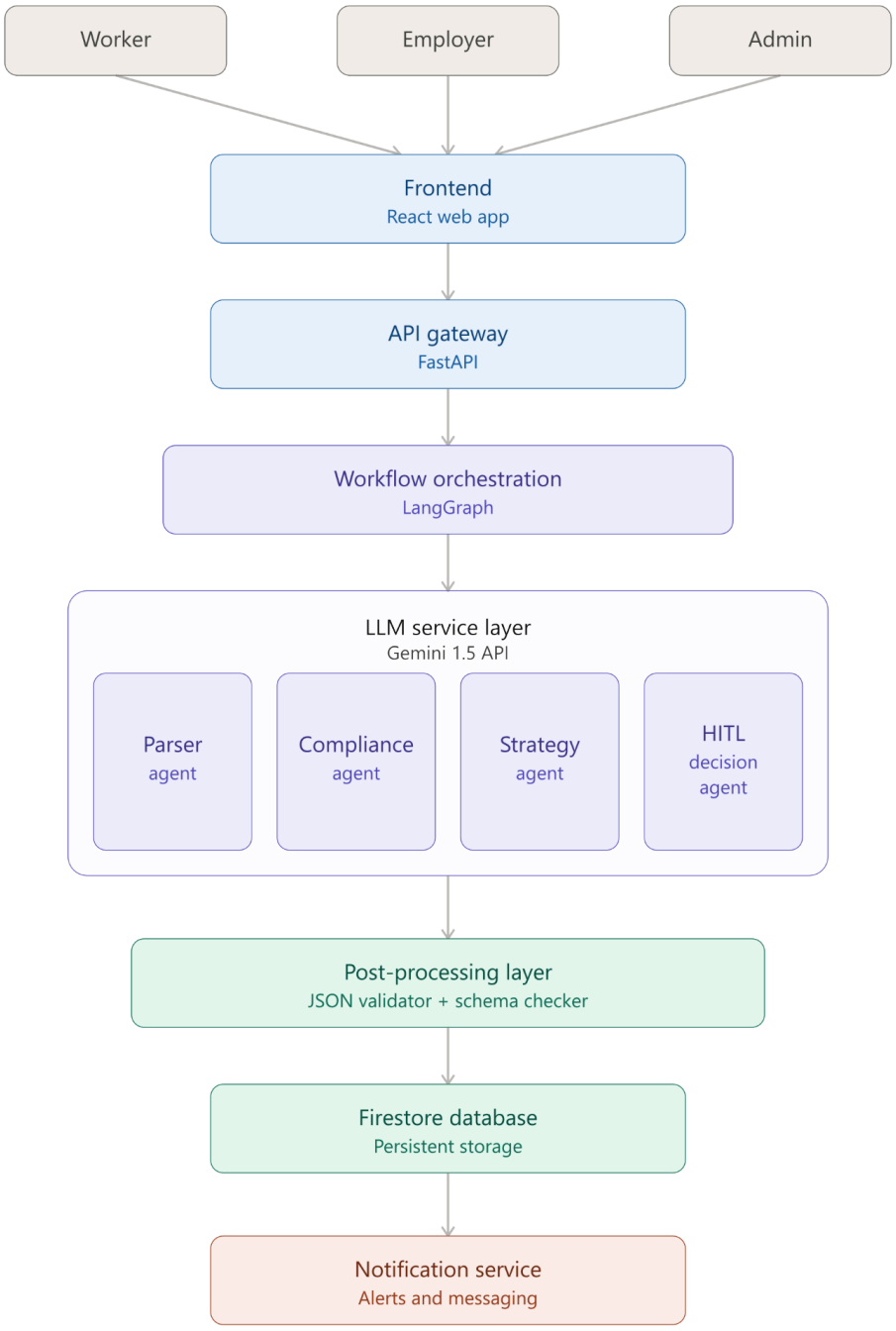
- Workflow Orchestration Service (LangGraph)
- Document Processing Service (OCR + extraction)
- Compliance Service (risk scoring + legal validation)
- Notification Service (alerts and updates)

All services are deployed on Google Cloud Platform, and share a centralized **Firestore database** for real-time state management.

## LLM as Service Layer

The architecture of PermitIQ integrates **Gemini 1.5 as a service layer**, not as a generic AI module.

### Dependency Diagram



## How Prompts Are Being Constructed and Sent to GLM API

Prompts are constructed dynamically inside each agent in the workflow engine.

Each prompt includes:

- OCR extracted document data (passport, medical report, licenses)
- Employer quota and company metadata
- Retrieved legal rules (via RAG system)
- Current workflow state from LangGraph
- Historical decision logs (compressed summaries)

The structured prompt is then sent to the Gemini 1.5 API via RESTful backend calls.

## What Goes Into the Context Window

The LLM context window includes:

- Chunked OCR outputs from documents
- Relevant regulatory clauses retrieved via RAG
- Current application workflow state
- Summarized historical decisions

Token optimization strategies:

- Document chunking before injection
- Retrieval-based filtering of legal content
- Context summarization of workflow history
- 24-hour caching of repeated inputs

## Response Handling & System Processing

LLM responses are returned in structured JSON format.

System processing flow:

1. JSON schema validation layer
2. Business rule validation layer
3. Workflow state transition engine (LangGraph)
4. Routing to:
    - Next agent OR
    - HITL module OR
    - Database storage layer

## Token Limitation Handling

- Pre-processing chunking of large documents
- RAG filtering for relevant legal clauses only
- Workflow state compression
- Caching of repeated document processing results

## Major API Calls Between System Components

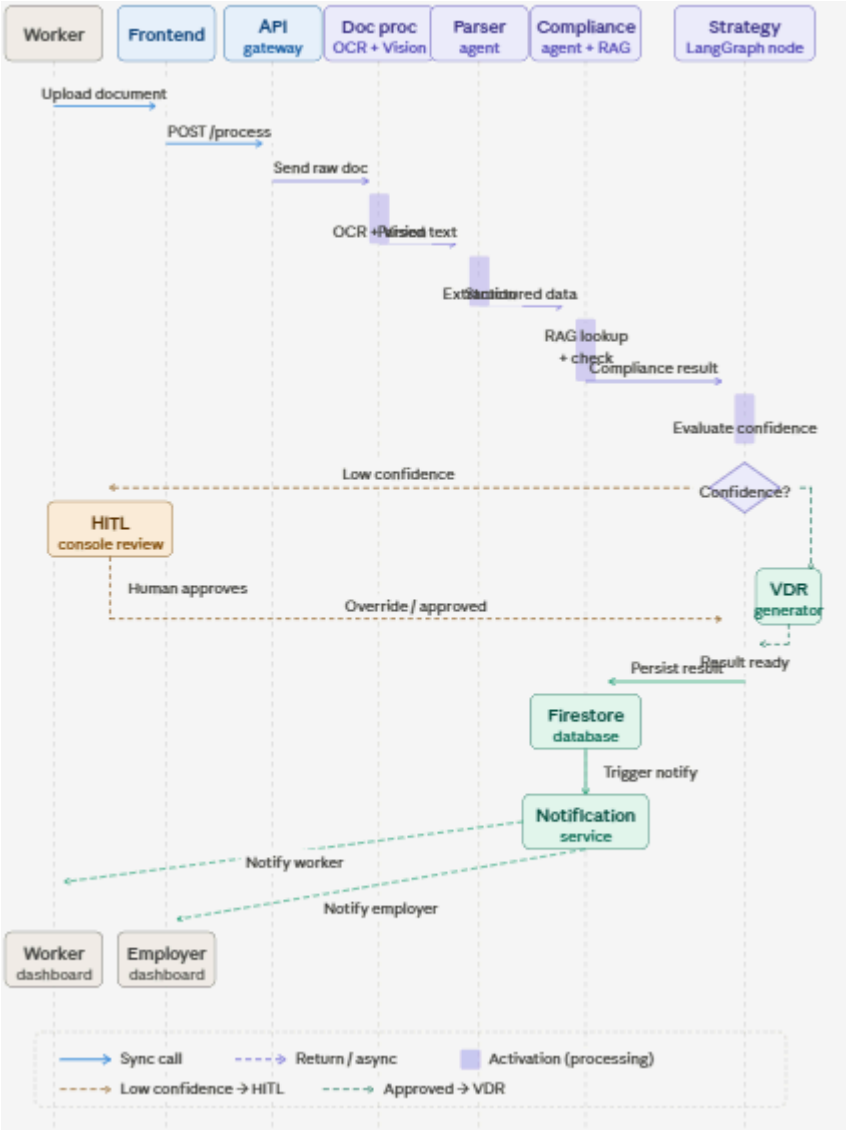
- Frontend → API Gateway: submit application request
- API Gateway → Document Service: OCR processing request
- Workflow Service → Gemini API: LLM reasoning request
- Workflow Service → Firestore: persist application state
- Notification Service → Frontend: status update event

## Sequence Diagram

## Use Case: Worker Application Submission Flow

## Step-by-step Interaction

1. Worker uploads documents via Web Application
2. Frontend sends request to API Gateway
3. API Gateway forwards request to Document Processing Service
4. Document Service performs OCR using Gemini Vision
5. Parser Agent extracts structured JSON data
6. Compliance Agent evaluates regulatory compliance
7. Strategy Agent determines next workflow step
8. If uncertainty detected → HITL Console triggered
9. If approved → VDR generation executed
10. Application state stored in Firestore
11. Notification Service updates Worker and Employer dashboards

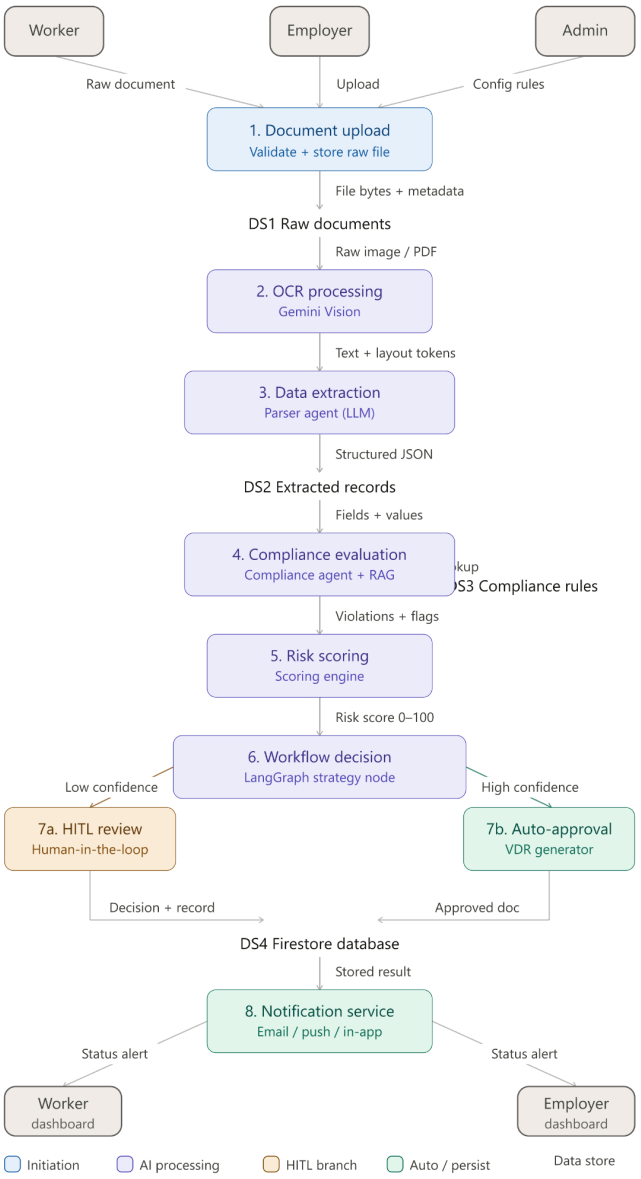

Worker
Frontend
API
gateway
Doc proc
OCR + Vision
Parser
agent
Compliance
agent + RAG
Strategy
LangGraph node
Upload document
POST /process
Send raw doc
OCR + Parsed text
Extracted data
RAG lookup
+ check
Compliance result
Evaluate confidence
Low confidence
Confidence?
HITL
console review
Human approves
Override / approved
VDR
generator
Result ready
Persist result
Firestore
database
Trigger notify
Notification
service
Notify worker
Notify employer
Worker
dashboard
Employer
dashboard
Sync call
Return / async
Activation (processing)
Low confidence → HITL
Approved → VDR

## Technological Stack

Frontend: React (Vite) + Tailwind CSS
Backend: FastAPI (Python)
Database: Firebase Firestore
Cloud/Deployment: Google Cloud Run
AI Model: Gemini 1.5 Pro / Flash
Workflow Engine: LangGraph

## Data Flow Diagram (DFD)

Illustrates system-wide data movement:



## Normalized Database Schema (3NF)

Worker

- worker_id (PK)
- passport_number
- nationality
- status

Employer

- employer_id (PK)
- company_name
- quota_limit

Application

- application_id (PK)
- worker_id (FK)
- employer_id (FK)
- status
- risk_score

Document

- document_id (PK)
- application_id (FK)
- document_type
- extracted_json

AuditLog

- log_id (PK)
- application_id (FK)
- agent_action
- timestamp

## Functional Requirements & Scope

## Minimum Viable Product

| # | Features | Description |
|---|---|---|
| 1 | Document Upload & Extraction | Users upload passport and documents, system extracts structured data using AI OCR |
| 2 | Compliance Risk Scoring | System evaluates application against regulatory rules and generates risk score |
| 3 | Workflow Automation | Multi-agent system determines application flow automatically |
| 4 | HITL Intervention | System triggers human review when AI confidence is low |
| 5 | VDR Generation | System generates structured visa documentation automatically |

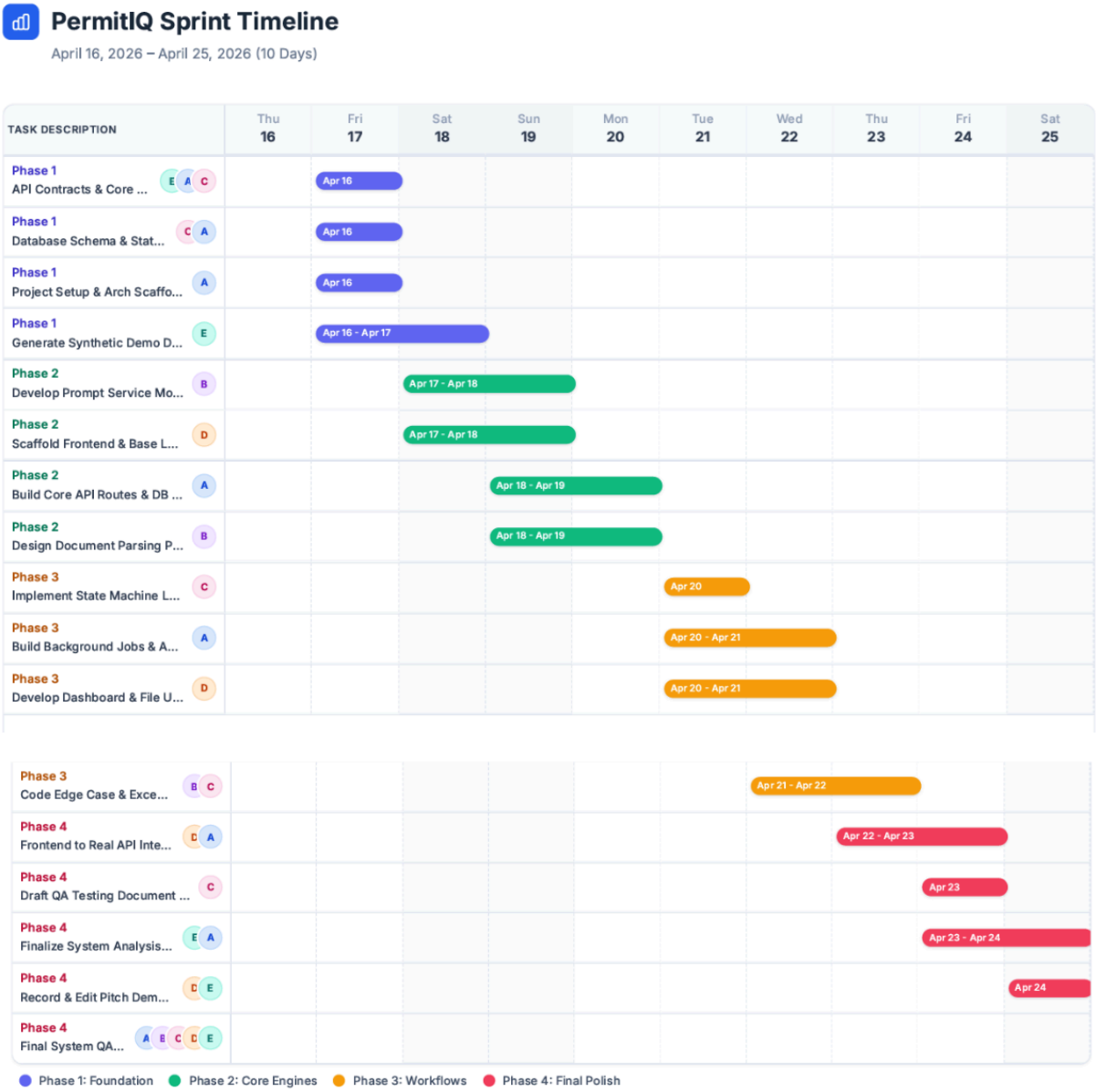
## Non-Functional Requirements (NFRs)

| Quality | Requirements | Implementation |
|---|---|---|
| Scalability | System must handle high concurrent application submissions | Auto-scaling Cloud Run services |
| Reliability | Application processing must ensure no lost state transitions | Firestore transactional updates |
| Maintainability | Each service must be independently deployable | Microservice architecture with CI/CD pipelines |
| Token Latency | LLM response must be within 3s (p95) | Async API + caching + timeout control |
| Cost Efficiency | Token usage must be optimized per workflow | Prompt truncation, chunking, and caching strategy |

## Out of Scope / Future Enhancements

- Real-time government API integration (JIM / JTKSM live systems)
- Mobile application (iOS/Android native app)
- Automated payment processing system
- Predictive hiring analytics and workforce forecasting
- Cross-country regulatory expansion beyond Malaysia

## Project Timeline




## Team Members Role

Member A: Long Ji Rui – Role Backend Lead – FastAPI Microservices, PostgreSQL Database Design & Async Background Jobs

Member B: Tan Shi Kai – Role AI/Prompt Engineer – GLM-4 API Integration, Prompt Service Module & JSON Schema Validation

Member C: Cheng Tze Xuan – Role Workflow Engine & QA – State Machine Logic, Compliance Calendar Generation & QA Testing Documentation

Member D: Tan Yin Xuan – Role Frontend Lead – React UI/UX, Employer Dashboard, Document Uploads & Exception UI

Member E: Prabu Natar – Role Integration & Documentation Lead – Project Management, API Contracts, SAD, PRD & Pitch Deck Submission